

D (枚举)

CF2144D

- 题意：有 N 种不同的商品，第 i 种商品的价格为 c_i ，每个商品上都贴着对应的价格标签。现在你决定搞促销活动，你会选一个统一的系数 $x \geq 2$ ，然后把第 i 种商品的价格变为 $\lceil \frac{c_i}{x} \rceil$ 。接着，你需要把每件上面的标签更新为正确的值，你可以利用原有的 N 个标签，如果没有所需要的标签，你就需要花费 y 元来打印新的标签。你得到的收入 = 商品总价格 - 打印标签的花费，请你算算最大可能的收入为多少
- 范围： $N \in [1, 2e5]$, $y \in [1, 1e9]$, $c_i \in [1, 2e5]$
- 关键思考：
 - 本题为决策类题目，常用思考方式为：**暴力** $\rightarrow$ DP || 贪心 || 网络流，常用优化手段为：二分、**前缀和**、双指针、数据结构
 - 首先观察题目性质，我们设 $mx = \max(c)$, $mx \in [1, 2e5]$ ，不难发现，当 x 超过 mx 之后，结果与 $x = mx$ 时完全相同，因此 x 的有效范围是 $[2, \max(2, mx)]$ ，我们可以考虑枚举 x
 - 对一个固定的 x 来说， $c_i \in [(i-1)x + 1, i * x]$ 的结果都是 i ，我们接下来研究 i 的取值范围：
 - 显然， i 的最小值是 1
 - $(i-1)x + 1 \leq mx$ ，故 $i \leq \lfloor \frac{mx-1}{x} \rfloor + 1$
 - $i \in [1, \lfloor \frac{mx-1}{x} \rfloor + 1]$
 - 由此可得，枚举 x 并枚举在系数 x 下的所有有效价格，这件事的时间复杂度为 $O(V \log V)$
 - 我们还需要快速计算出收入，收入分为两部分：
 - 商品总价格：贡献法，枚举价格 i 的时候，我们希望知道最终会有多少个商品的价格变为 i ，即得到 now_cnt_i 。我们可以在值域上做前缀和，定义 $pre[i] :=$ 原价格 $\leq i$ 的商品数量。用 $pre[i * x] - pre[(i-1)x]$ 即可得到 now_cnt_i 。
 - 打印标签：贡献法，枚举价格 i 的时候，我们希望知道初始有多少个价格为 i 的标签，即 pre_cnt_i ，则 $\max(0, now_cnt_i - pre_cnt_i)$ 就是需要打印的标签数量。

答案为

$$\max \left(\sum_{i=1}^{\lfloor \frac{mx-1}{x} \rfloor + 1} i \times now_cnt_i - need \times y \right)$$

- 算法总复杂度为 $O(V \log V)$

E1 (DP + 组合数学)

CF2144E1

- 题意：有 N 座塔排成一排，第 i 座塔的高度为 h_i 。从左边看，你能看到所有比之前所有塔都高的塔，从右边看同理。形式化地，设 $L(h) :=$ 从左边看到的塔高度集合， $R(h) :=$ 从右边看到的塔高度集合，则有：
 - $L(h) = \{h_i | \forall j < i, h_j < h_i\}$
 - $R(h) = \{h_i | \forall j > i, h_j < h_i\}$

定义 h 的子序列 h' 是合法的，当且仅当 $L(h') = L(h) \wedge R(h') = R(h)$

求 h 的合法子序列数量，结果对998244353取模

- 范围： $N \in [1, 5000]$, $h_i \in [1, 1e9]$

• 关键思考：

- 本题为计数类题目，常用思考方式为：暴力→DP||组合数学，常用优化手段为：前缀和、双指针、数学
- 首先观察题目性质，这是经典的**双向可见性**问题，这种问题的核心在于 $\max(h)$ ，设 $h_i = \max(h)$ ，则 h_i 会把数组分割为3段，即 $[1, h_i - 1][h_i][h_i + 1, N]$ ，这样分割的好处在于：我们把对 $L(h)$ 和 $R(h)$ 的分析完全独立开了
- 设 $L(h)$ 的长度为 a ， $R(h)$ 的长度为 b
- 则 h' 是合法子序列的充要条件为： $L([1, h_i - 1])$ 是 $L(h)$ 的长度为 $a - 1$ 或 a 的前缀，并且 $R([h_i + 1, N])$ 是 $R(h)$ 的长度为 $b - 1$ 或 b 的前缀
- 如何进行计数呢？
 - 显然，我们只需要在每个 $h_i = \max(h)$ 的位置进行计数
 - 钦定 h_i 为选出的子序列中最后一个 $\max(h)$ ，这样相当于规定了 $L([1, h_i - 1])$ 是 $L(h)$ 的长度为 $a - 1$ 或 a 的前缀，但 $R([h_i + 1, N])$ 必须是 $R(h)$ 的长度为 $b - 1$ 的前缀
 - 进一步简化，我们设 $f[i] :=$ 以 h_i 结尾，前面选择的数都小于 h_i ，构成 $L(h)$ 的方案数。同理，设 $g[i] :=$ 以 h_i 开头，后面选择的数都小于 h_i ，构成 $R(h)$ 的方案数。
 - 不难发现， $f[i]$ 正是 $L([1, h_i - 1])$ 为 $L(h)$ 的长度为 $a - 1$ 的前缀的方案数， $g[i]$ 同理
 - 因此， $f[i] \times g[i]$ 就是 $(a - 1) \times (b - 1)$ 这种情况的方案数
 - 此外，我们还要求出 $a \times (b - 1)$ 这种情况的方案数，这就需要我们额外维护 $L([1, h_i - 1])$ 是 $L(h)$ 的长度为 a 的前缀的方案数，我们用一个变量 sum 进行维护：
 - 这样， $sum \times g[i]$ 就是 $a \times (b - 1)$ 这种情况的方案数
 - 下面考虑如何维护 sum ：
 - 由于 $L(h)$ 的第 a 项（最后一项）是 $\max(h)$ ，因此对任意的 h_i 都有**选或不选**两种方案，对应于：
 - $sum = sum \times 2$
 - 当 $h_i = \max(h)$ 时，长度为 a 的前缀可以由长度为 $a - 1$ 的前缀拼接 h_i 得到，对应于：
 - $sum = sum + f[i]$
- 回到核心问题，如何求出 $f[i]$ 和 $g[i]$ ？这就变成一个典型的线性DP问题，下面以 $f[i]$ 为例：
 - 首先求出 $L(h)$ ，设其长度为 a
 - 定义 $dp[i][j] :=$ 考虑前 i 个字符，组成长度为 j 的前缀的方案数
 - 状态转移：
 - 当 $h_i \leq L_j$ 时，我们可以将 h_i 拼接在后面，也可以不拼接，即**选或不选**：
 - $dp[i][j] = 2 \times dp[i - 1][j]$
 - 当 $h_i = L_j$ 时：
 - 若 $j = 1$ ，此时 h_i 可以单独作为一个 L_1 子序列：
 - $dp[i][j] = dp[i][j] + 1$
 - 若 $j \geq 2$ ，我们可以将 h_i 拼接在 L_{j-1} 后面，即：
 - $dp[i][j] = dp[i][j] + dp[i - 1][j - 1]$

- 当 $f[i] = \max(h)$ 时, 我们更新:
 - $f[i] = dp[i - 1][a - 1]$
- 由于 $f[i]$ 和 $g[i]$ 的计算逻辑相同, 只不过一个是正向遍历数组, 一个是反向遍历数组, 因此我们可以通过 `reverse` 操作将两种情况统一起来, 简化代码逻辑
- 时间复杂度为 $O(N^2)$

E2 (二分 + 线段树优化DP)

[CF2144E2](#)

- 题意: 同 E1
- 范围: $N \in [1, 3e5], h_i \in [1, 1e9]$
- 关键思考:
 - E1 的朴素实现是 $O(N^2)$ 的, 无法通过本题, 我们发现瓶颈在于求解 $f[i]$ 和 $g[i]$
 - 观察转移方程:
 - $dp[i][j] = 2 \times dp[i - 1][j]$, if $L_j \geq h_i$
 - 我们压缩掉第一维, 实际就变为了 $dp[j] = 2 \times dp[j - 1]$, 这可以看作是 **区间乘法**
 - $dp[i][j] = dp[i][j] + dp[i - 1][j - 1]$, if $L_j = h_i$
 - 压缩掉第一维, 就变为了 $dp[j] = dp[j] + dp[j - 1]$, 这可以看作是 **单点查询 + 单点修改**
 - $f[i] = dp[i - 1][a - 1]$:
 - 压缩掉第一维, 就变为了 $f[i] = dp[a - 1]$, 这可以看作是 **单点查询**
 - 因此, 我们需要一个支持 **区间乘法 + 单点查询 + 单点修改** 的数据结构, 线段树正是不二之选